

# Matchings of the period-3 pendant ladder: a mod-3 interlacing recurrence and explicit modular residue tables

(Lean 4 machine-checked claim of record)

Aurora0134\*

September 29, 2026

## Abstract

Let  $a(n)$  be the number of matchings (the empty one included) of the period-3 pendant ladder: the ladder graph  $P_n \square K_2$  with one pendant leaf attached to the upper-rail vertex of every column  $3k+1$ . The AI4Math pipeline formalised and machine-proved three exact laws of the recurrence-defined counterpart `ap3` of  $a$  (with `ap3 n = a(n)` under the 0-based convention, the base term  $a(0) = 1$  included): (1) a mod-3 interlacing identity that splits the order-12 sparse recurrence into three order-4 subsequences with the same coefficients  $(45, 63, 11, -1)$ ; (2) an explicit period-15 residue table for `ap3 mod 2`; (3) an explicit period-30 residue table for `ap3 mod 4`. All three theorems compile with no `sorry`, and `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. Two independent novelty reviews found no occupying record for the sequence, its recurrence, or its three mod-3 subsequences; the plain ladder, the period-1 pendant (corona) and the period-2 pendant relatives are registered, period 3 is not. The claim is strictly *new sequence / new recurrence*, never new mathematics. This note is a priority claim of record, not a full paper.

## 1 Introduction

Three theorems about one integer sequence are reported here. The sequence counts all matchings of the period-3 pendant ladder; its first val-

---

\*Contact: via the GitHub account Aurora0134; ORCID placeholder — to be filled in by the author before any upload. No personal information is carried in this note.

ues are  $a(0) = 1$  (the empty graph with its empty matching) and, for  $n \geq 1$ , 3, 10, 32, 150, 457, 1489, 6948, 21191, 69032, 322142, ..., and the conjectured law recorded with the object was the sparse recurrence  $a(n) = 45a(n-3) + 63a(n-6) + 11a(n-9) - a(n-12)$ , equivalently: the three mod 3 subsequences all obey the same order-4 recurrence with coefficients (45, 63, 11, -1). The proofs were produced by the AI4Math pipeline, in which LLM workers generate statements and proof scripts and the Lean 4 kernel is the sole adjudicator at every gate: statements are frozen by hash before proving, and a separate audit re-computes the symmetric difference, scans the whole file for `sorry/admit` including comments, and re-prints the axiom dependencies [1]. The kernel layer proves the pure-sequence statements (definitions below are initial-values-plus-recurrence,  $\mathbb{N} \rightarrow \mathbb{Z}$ , no graph objects); the identification of the recurrence-defined sequence with the matching count of the pendant ladder is a conjecture layer carried by the pipeline’s computational evidence (five methodologically distinct counting implementations in agreement over 241 terms, double-window modular scans) and is not kernel-proved; Section 6 states this boundary verbatim. This note is a priority claim of record deposited so that a dated, citable public record exists ahead of the narrative paper; no literature review is attempted here.

## 2 Statement

Throughout, **ap3** is the main column: an  $\mathbb{N} \rightarrow \mathbb{Z}$  sequence with twelve frozen initial values and the order-12 sparse recurrence  $\mathbf{ap3}(n+12) = 45\mathbf{ap3}(n+9) + 63\mathbf{ap3}(n+6) + 11\mathbf{ap3}(n+3) - \mathbf{ap3}n$ . The index convention is fixed by the pipeline’s claims record:  $\mathbf{ap3}n = a(n)$  for all  $n \geq 0$  (0-based; the base term  $a(0) = 1$ , the empty graph, included), the twelve frozen initial values are the 0-based  $a(0..11)$ , and **bp0**, **bp1**, **bp2** are the three mod 3 subsequences,  $\mathbf{bp}_r k = a(3k+r)$  (0-based), each with four initial values (the **ap3** values at indices  $3k+r$ ) and the order-4 recurrence  $\mathbf{bp}_r(k+4) = 45\mathbf{bp}_r(k+3) + 63\mathbf{bp}_r(k+2) + 11\mathbf{bp}_r(k+1) - \mathbf{bp}_r k$ . **pat15** and **pat30** are the explicit residue tables:  $\mathbb{N} \rightarrow \mathbb{Z}$  functions equal to 0 outside the listed classes.

**Theorem 1** (mod-3 interlacing identity). *For every  $k : \mathbb{N}$ ,*

$$\mathbf{ap3}(3k) = \mathbf{bp0}k \wedge \mathbf{ap3}(3k+1) = \mathbf{bp1}k \wedge \mathbf{ap3}(3k+2) = \mathbf{bp2}k.$$

*That is: the order-12 sparse recurrence decouples along  $n \bmod 3$  into three order-4 subsequences that share the coefficient vector (45, 63, 11, -1).*

**Theorem 2** (explicit period-15 residue table for mod 2). *For all  $n, r : \mathbb{N}$  with  $n \bmod 15 = r$ ,  $\text{ap3 } n \bmod 2 = \text{pat15 } r$ , where **pat15** is the explicit table  $(1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0)$  on the fifteen residue classes; equivalently  $\text{ap3 } n$  is odd exactly when  $n \bmod 15 \in \{0, 1, 5, 6, 8, 13\}$ .*

**Theorem 3** (explicit period-30 residue table for mod 4). *For all  $n, r : \mathbb{N}$  with  $n \bmod 30 = r$ ,  $\text{ap3 } n \bmod 4 = \text{pat30 } r$ , where **pat30** is the explicit thirty-entry table reproduced in the listing below.*

Listing 1: Frozen base definitions and the head of Theorem 1, verbatim lines 37–89 of `formalized/comb10-ladder-pend3.statement.txt` (sha256 47d54db7): the four sequence definitions and the interlacing statement.

```

1  /-- **序列 ap3 (主列, 12 阶稀疏递推) **.
2  初值 (12 项, 0-based a(0..11)) : 1, 3, 10, 32, 150, 457, 1489,
   6948, 21191, 69032, 322142, 982496;
3  递推 ap3 (n+12) = 45·ap3 (n+9) + 63·ap3 (n+6) + 11·ap3 (n+3) - ap3
   n (n ≥ 0) 。 -/
4  def ap3 : ℕ → ℤ
5    | 0 => 1
6    | 1 => 3
7    | 2 => 10
8    | 3 => 32
9    | 4 => 150
10   | 5 => 457
11   | 6 => 1489
12   | 7 => 6948
13   | 8 => 21191
14   | 9 => 69032
15   | 10 => 322142
16   | 11 => 982496
17   | n + 12 => 45 * ap3 (n + 9) + 63 * ap3 (n + 6) + 11 * ap3 (n +
   3) - ap3 n
18
19 /-- **序列 bp0 (mod-3 子列 r = 0, 四阶递推) **.
20 初值 = ap3 在下标 0, 3, 6, 9 处之值: 1, 32, 1489, 69032;
21 递推 bp0 (k+4) = 45·bp0 (k+3) + 63·bp0 (k+2) + 11·bp0 (k+1) - bp0 k
   (k ≥ 0) 。 -/
22 def bp0 : ℕ → ℤ
23   | 0 => 1
24   | 1 => 32
25   | 2 => 1489
26   | 3 => 69032
27   | k + 4 => 45 * bp0 (k + 3) + 63 * bp0 (k + 2) + 11 * bp0 (k +
   1) - bp0 k

```

```

28
29 /-- **序列  $bp1$  ( $mod-3$  子列  $r = 1$ , 四阶递推) **.
30 初值 =  $ap3$  在下标  $1, 4, 7, 10$  处之值:  $3, 150, 6948, 322142$ ;
31 递推  $bp1(k+4) = 45 \cdot bp1(k+3) + 63 \cdot bp1(k+2) + 11 \cdot bp1(k+1) - bp1(k)$ 
   ( $k \geq 0$ )。 -/
32 def bp1 :  $\mathbb{N} \rightarrow \mathbb{Z}$ 
33   | 0 => 3
34   | 1 => 150
35   | 2 => 6948
36   | 3 => 322142
37   | k + 4 => 45 * bp1 (k + 3) + 63 * bp1 (k + 2) + 11 * bp1 (k +
   1) - bp1 k
38
39 /-- **序列  $bp2$  ( $mod-3$  子列  $r = 2$ , 四阶递推) **.
40 初值 =  $ap3$  在下标  $2, 5, 8, 11$  处之值:  $10, 457, 21191, 982496$ ;
41 递推  $bp2(k+4) = 45 \cdot bp2(k+3) + 63 \cdot bp2(k+2) + 11 \cdot bp2(k+1) - bp2(k)$ 
   ( $k \geq 0$ )。 -/
42 def bp2 :  $\mathbb{N} \rightarrow \mathbb{Z}$ 
43   | 0 => 10
44   | 1 => 457
45   | 2 => 21191
46   | 3 => 982496
47   | k + 4 => 45 * bp2 (k + 3) + 63 * bp2 (k + 2) + 11 * bp2 (k +
   1) - bp2 k
48
49 /-- ** $T1$  (主攻) **: 主列  $12$  阶稀疏递推 = 三条同系数四阶子列按  $n \bmod 3$ 
   交织——
50 对一切  $k$ ,  $ap3(3k) = bp0\ k$ 、 $ap3(3k+1) = bp1\ k$ 、 $ap3(3k+2) = bp2\ k$ 
   。
51 (claims.md § $T1$ : 递推只含  $3$  的倍数滞后, 三条  $mod-3$  副列天然解耦, 无
   需辅助  $12$  阶恒等式引理。) -/
52 theorem ap3_interleave (k :  $\mathbb{N}$ ) :
53    $ap3(3 * k) = bp0\ k \wedge ap3(3 * k + 1) = bp1\ k \wedge ap3(3 * k +$ 
    $2) = bp2\ k := by$ 

```

Listing 2: Frozen definition of `pat15` and the head of Theorem 2, verbatim lines 92–116 of the frozen statement file.

```

1 /-- **余数样式  $pat15$ ** ( $15$  留数类  $\mapsto mod\ 2$  余数) :
2  $0 \mapsto 1, 1 \mapsto 1, 2 \mapsto 0, 3 \mapsto 0, 4 \mapsto 0, 5 \mapsto 1, 6 \mapsto 1, 7 \mapsto 0, 8 \mapsto 1, 9 \mapsto 0,$ 
3  $10 \mapsto 0, 11 \mapsto 0, 12 \mapsto 0, 13 \mapsto 1$ , 其余  $\mapsto 0$ 
4 (即  $ap3\ n$  为奇  $\iff n \bmod 15 \in \{0, 1, 5, 6, 8, 13\}$ )。 -/
5 def pat15 :  $\mathbb{N} \rightarrow \mathbb{Z}$ 
6   | 0 => 1
7   | 1 => 1

```

```

8 | 2 => 0
9 | 3 => 0
10 | 4 => 0
11 | 5 => 1
12 | 6 => 1
13 | 7 => 0
14 | 8 => 1
15 | 9 => 0
16 | 10 => 0
17 | 11 => 0
18 | 12 => 0
19 | 13 => 1
20 | _ => 0
21
22 /-- **T2 (伴随) **: ap3 的 mod 2 余数仅依赖 n mod 15, 样式为 pat15
23 (0-based 口径; claims.md ST2 样式 (1,1,0,0,0,1,1,0,1,0,0,0,0,1,0)
24 )。 -/
25 theorem ap3_mod2_period15 (n r : ℕ) (hr : n % 15 = r) :
    ap3 n % 2 = pat15 r := by

```

Listing 3: Frozen definition of `pat30` and the head of Theorem 3, verbatim lines 119–159 of the frozen statement file.

```

1 /-- **余数样式 pat30** (30 留数类  $\mapsto$  mod 4 余数) :
2 0 $\mapsto$ 1, 1 $\mapsto$ 3, 2 $\mapsto$ 2, 3 $\mapsto$ 0, 4 $\mapsto$ 2, 5 $\mapsto$ 1, 6 $\mapsto$ 1, 7 $\mapsto$ 0, 8 $\mapsto$ 3, 9 $\mapsto$ 0,
3 10 $\mapsto$ 2, 11 $\mapsto$ 0, 12 $\mapsto$ 2, 13 $\mapsto$ 1, 14 $\mapsto$ 2, 15 $\mapsto$ 1, 16 $\mapsto$ 1, 17 $\mapsto$ 2, 18 $\mapsto$ 2, 19 $\mapsto$ 
4 2,
5 20 $\mapsto$ 1, 21 $\mapsto$ 3, 22 $\mapsto$ 2, 23 $\mapsto$ 1, 24 $\mapsto$ 2, 25 $\mapsto$ 2, 26 $\mapsto$ 0, 27 $\mapsto$ 0, 28 $\mapsto$ 1, 其余
6  $\mapsto$  0。 -/
7 def pat30 : ℕ  $\rightarrow$  ℤ
8 | 0 => 1
9 | 1 => 3
10 | 2 => 2
11 | 3 => 0
12 | 4 => 2
13 | 5 => 1
14 | 6 => 1
15 | 7 => 0
16 | 8 => 3
17 | 9 => 0
18 | 10 => 2
19 | 11 => 0
20 | 12 => 2
21 | 13 => 1
22 | 14 => 2

```

```

21 | 15 => 1
22 | 16 => 1
23 | 17 => 2
24 | 18 => 2
25 | 19 => 2
26 | 20 => 1
27 | 21 => 3
28 | 22 => 2
29 | 23 => 1
30 | 24 => 2
31 | 25 => 2
32 | 26 => 0
33 | 27 => 0
34 | 28 => 1
35 | _ => 0
36
37 /-- **T3 (伴随) **: ap3 的 mod 4 余数仅依赖 n mod 30, 样式为 pat30
38 (蕴含 T2: 奇值类 mod 30 = {0, 1, 5, 6, 8, 13, 15, 16, 20, 21, 23,
39 28}, 每类 mod 2 与 pat15 一致;
40 蕴含关系归证明侧, statement 层 T2/T3 并列独立)。 -/
41 theorem ap3_mod4_period30 (n r : ℕ) (hr : n % 30 = r) :
    ap3 n % 4 = pat30 r := by

```

The complete final file is reproduced verbatim in Section 3 (whole file, no excerpting); it also ships in this deposit under **proofs/**. The frozen statement file carries the three theorem bodies as **sorry** placeholders (that is its role: the formalisation department freezes statements, the proving department replaces bodies); the excerpts above stop at each theorem head, so no **sorry** line enters any listing, and the final proof file has none anywhere.

### 3 Proof

All three proofs are strong inductions on the index, with the recursion depth 12 (the sparse recurrence's gap) between the induction hypothesis instances used and the conclusion. For Theorem 1 the decoupling is structural: the sparse recurrence only lags by multiples of 3, so each mod 3 subsequence satisfies the order-4 recurrence on its own; the proof first establishes the three one-sided lemmas  $\text{ap3}(3k+r) = \text{bp}_r k$  ( $r = 0, 1, 2$ ) by strong induction (twelve base cases discharged by case enumeration and arithmetic normalisation; induction step: unfold  $\text{bp}_r$ , rewrite the four occurrences through the induction hypotheses, and match the  $\text{ap3}$  recurrence instance at index  $3k+r$ ), then assembles the conjunction. Theorems 2 and 3 prove the full-column

residue laws directly: a **period** lemma ( $\forall m, \text{ap3 } m \bmod q = \text{pat } (m \bmod P)$ ) is established by strong induction, where the induction step rewrites the recurrence modulo  $q$  (for  $q = 2$ : all four coefficients are 1; for  $q = 4$ : 1, 3, 3, 3), enumerates the  $P$  residue classes of the induction target  $m \bmod P$  ( $P = 15$ , resp. 30), shifts each of the four lagged terms into a decided residue class, and closes each case by table lookup and arithmetic; the outer theorem instantiates the **period** lemma at  $m = n$ . The third theorem's file carries a **set\_option maxHeartbeats 800000** decoration: an elaboration-resource knob for the thirty-case enumeration, with no effect on the statement or the kernel semantics. The complete final file follows verbatim (440 lines, no excerpting; the working copy is CRLF, listing LF-normalised per the chorded-cycle precedent, byte-exact original in **proofs/**).

Listing 4: Complete proof file of Theorems 1, 2 and 3, verbatim lines 1–440 (whole file) of **final/comb10-ladder-pend3-proved.lean** (sha256 e56e496f).

```

1  /-
2    AI4Math 流水线 · AI 生成 · 2026-09-29
3    任务线: comb10-ladder-pend3 (pool-comb-10) · 三定理合并终审稿
4    冻结快照: formalized/comb10-ladder-pend3.statement.txt
5      (sha256 47
6        d54db7c3d5309720537295055d5869e990618399950aaacb311f8028a9bba9)
7    来源分件 (kernel 已逐件过): attempts/comb10-t1.lean / comb10-t2.lean /
8      comb10-t3.lean
9    声明层对冻结件零改动; T3 前一条 set_option maxHeartbeats 800000 为编译资
10   源旋钮
11   (comb-09 T2 同类判例, 内核语义不受影响)。
12 -/
13
14 import Mathlib
15
16 def ap3 : ℕ → ℤ
17   | 0 => 1
18   | 1 => 3
19   | 2 => 10
20   | 3 => 32
21   | 4 => 150
22   | 5 => 457
23   | 6 => 1489
24   | 7 => 6948
25   | 8 => 21191
26   | 9 => 69032
27   | 10 => 322142
28   | 11 => 982496
29   | n + 12 => 45 * ap3 (n + 9) + 63 * ap3 (n + 6) + 11 * ap3 (n + 3) -

```

```

27     ap3 n
28 def bp0 : ℕ → ℤ
29   | 0 => 1
30   | 1 => 32
31   | 2 => 1489
32   | 3 => 69032
33   | k + 4 => 45 * bp0 (k + 3) + 63 * bp0 (k + 2) + 11 * bp0 (k + 1) - bp0
      k
34
35 def bp1 : ℕ → ℤ
36   | 0 => 3
37   | 1 => 150
38   | 2 => 6948
39   | 3 => 322142
40   | k + 4 => 45 * bp1 (k + 3) + 63 * bp1 (k + 2) + 11 * bp1 (k + 1) - bp1
      k
41
42 def bp2 : ℕ → ℤ
43   | 0 => 10
44   | 1 => 457
45   | 2 => 21191
46   | 3 => 982496
47   | k + 4 => 45 * bp2 (k + 3) + 63 * bp2 (k + 2) + 11 * bp2 (k + 1) - bp2
      k
48
49 theorem ap3_sub_0 (k : ℕ) : ap3 (3 * k) = bp0 k := by
50   induction k using Nat.strong_induction_on with
51   | h k ih =>
52     by_cases hk : k < 4
53     · interval_cases k <|> norm_num [ap3, bp0]
54     · let t := k - 4
55       have ht : k = t + 4 := by dsimp [t]; omega
56       have h0 := ih t (by omega)
57       have h1 := ih (t + 1) (by omega)
58       have h2 := ih (t + 2) (by omega)
59       have h3 := ih (t + 3) (by omega)
60       rw [ht, bp0]
61       have hidx : 3 * (t + 4) = 3 * t + 12 := by omega
62       rw [hidx]
63       have hrec : ap3 (3 * t + 12) =
64         45 * ap3 (3 * t + 9) + 63 * ap3 (3 * t + 6) +
65         11 * ap3 (3 * t + 3) - ap3 (3 * t) := by
66         rw [ap3]
67         rw [hrec]
68         have e3 : 3 * t + 9 = 3 * (t + 3) := by omega
69         have e2 : 3 * t + 6 = 3 * (t + 2) := by omega
70         have e1 : 3 * t + 3 = 3 * (t + 1) := by omega
71         rw [e3, e2, e1, h3, h2, h1, h0]

```



```

72
73 theorem ap3_sub_1 (k : ℕ) : ap3 (3 * k + 1) = bp1 k := by
74   induction k using Nat.strong_induction_on with
75   | h k ih =>
76     by_cases hk : k < 4
77     · interval_cases k <|> norm_num [ap3, bp1]
78     · let t := k - 4
79       have ht : k = t + 4 := by dsimp [t]; omega
80       have h0 := ih t (by omega)
81       have h1 := ih (t + 1) (by omega)
82       have h2 := ih (t + 2) (by omega)
83       have h3 := ih (t + 3) (by omega)
84       rw [ht, bp1]
85       have hidx : 3 * (t + 4) + 1 = (3 * t + 1) + 12 := by omega
86       rw [hidx]
87       have hrec : ap3 ((3 * t + 1) + 12) =
88         45 * ap3 ((3 * t + 1) + 9) + 63 * ap3 ((3 * t + 1) + 6) +
89         11 * ap3 ((3 * t + 1) + 3) - ap3 (3 * t + 1) := by
90         rw [ap3]
91         rw [hrec]
92         have e3 : (3 * t + 1) + 9 = 3 * (t + 3) + 1 := by omega
93         have e2 : (3 * t + 1) + 6 = 3 * (t + 2) + 1 := by omega
94         have e1 : (3 * t + 1) + 3 = 3 * (t + 1) + 1 := by omega
95         rw [e3, e2, e1, h3, h2, h1, h0]
96
97 theorem ap3_sub_2 (k : ℕ) : ap3 (3 * k + 2) = bp2 k := by
98   induction k using Nat.strong_induction_on with
99   | h k ih =>
100     by_cases hk : k < 4
101     · interval_cases k <|> norm_num [ap3, bp2]
102     · let t := k - 4
103       have ht : k = t + 4 := by dsimp [t]; omega
104       have h0 := ih t (by omega)
105       have h1 := ih (t + 1) (by omega)
106       have h2 := ih (t + 2) (by omega)
107       have h3 := ih (t + 3) (by omega)
108       rw [ht, bp2]
109       have hidx : 3 * (t + 4) + 2 = (3 * t + 2) + 12 := by omega
110       rw [hidx]
111       have hrec : ap3 ((3 * t + 2) + 12) =
112         45 * ap3 ((3 * t + 2) + 9) + 63 * ap3 ((3 * t + 2) + 6) +
113         11 * ap3 ((3 * t + 2) + 3) - ap3 (3 * t + 2) := by
114         rw [ap3]
115         rw [hrec]
116         have e3 : (3 * t + 2) + 9 = 3 * (t + 3) + 2 := by omega
117         have e2 : (3 * t + 2) + 6 = 3 * (t + 2) + 2 := by omega
118         have e1 : (3 * t + 2) + 3 = 3 * (t + 1) + 2 := by omega
119         rw [e3, e2, e1, h3, h2, h1, h0]
120

```

```

121 theorem ap3_interleave (k : ℕ) :
122   ap3 (3 * k) = bp0 k ∧ ap3 (3 * k + 1) = bp1 k ∧ ap3 (3 * k + 2) = bp2
123   k := by
124   exact ⟨ap3_sub_0 k, ap3_sub_1 k, ap3_sub_2 k⟩
125
126 def pat15 : ℕ → ℤ
127 | 0 => 1
128 | 1 => 1
129 | 2 => 0
130 | 3 => 0
131 | 4 => 0
132 | 5 => 1
133 | 6 => 1
134 | 7 => 0
135 | 8 => 1
136 | 9 => 0
137 | 10 => 0
138 | 11 => 0
139 | 12 => 0
140 | 13 => 1
141 | _ => 0
142
143 theorem ap3_mod2_period15 (n r : ℕ) (hr : n % 15 = r) :
144   ap3 n % 2 = pat15 r := by
145   have hperiod : ∀ m : ℕ, ap3 m % 2 = pat15 (m % 15) := by
146     intro m
147     induction m using Nat.strong_induction_on with
148     | h m ih =>
149       by_cases hm : m < 12
150       · interval_cases m <|> decide
151       · let t := m - 12
152         have hmt : m = t + 12 := by
153           dsimp [t]
154           omega
155         rw [hmt, ap3]
156         have h0 := ih t (by omega)
157         have h3 := ih (t + 3) (by omega)
158         have h6 := ih (t + 6) (by omega)
159         have h9 := ih (t + 9) (by omega)
160         have hresBound : t % 15 < 15 := Nat.mod_lt _ (by omega)
161         interval_cases hres : t % 15
162         case «0» =>
163           rw [(show (t + 3) % 15 = 3 by omega), (show (t + 6) % 15 = 6 by
164             omega),
165             (show (t + 9) % 15 = 9 by omega), (show (t + 12) % 15 = 12
166               by omega)] at *
167           norm_num [pat15] at *
168           omega
169         case «1» =>

```

```

167         rw [(show (t + 3) % 15 = 4 by omega), (show (t + 6) % 15 = 7 by
168         omega),
169         (show (t + 9) % 15 = 10 by omega), (show (t + 12) % 15 = 13
170         by omega)] at *
171         norm_num [pat15] at *
172         omega
173     case «2» =>
174         rw [(show (t + 3) % 15 = 5 by omega), (show (t + 6) % 15 = 8 by
175         omega),
176         (show (t + 9) % 15 = 11 by omega), (show (t + 12) % 15 = 14
177         by omega)] at *
178         norm_num [pat15] at *
179         omega
180     case «3» =>
181         rw [(show (t + 3) % 15 = 6 by omega), (show (t + 6) % 15 = 9 by
182         omega),
183         (show (t + 9) % 15 = 12 by omega), (show (t + 12) % 15 = 0
184         by omega)] at *
185         norm_num [pat15] at *
186         omega
187     case «4» =>
188         rw [(show (t + 3) % 15 = 7 by omega), (show (t + 6) % 15 = 10
189         by omega),
190         (show (t + 9) % 15 = 13 by omega), (show (t + 12) % 15 = 1
191         by omega)] at *
192         norm_num [pat15] at *
193         omega
194     case «5» =>
195         rw [(show (t + 3) % 15 = 8 by omega), (show (t + 6) % 15 = 11
196         by omega),
197         (show (t + 9) % 15 = 14 by omega), (show (t + 12) % 15 = 2
198         by omega)] at *
199         norm_num [pat15] at *
200         omega
201     case «6» =>
202         rw [(show (t + 3) % 15 = 9 by omega), (show (t + 6) % 15 = 12
203         by omega),
204         (show (t + 9) % 15 = 0 by omega), (show (t + 12) % 15 = 3
205         by omega)] at *
206         norm_num [pat15] at *
207         omega
208     case «7» =>
209         rw [(show (t + 3) % 15 = 10 by omega), (show (t + 6) % 15 = 13
210         by omega),
211         (show (t + 9) % 15 = 1 by omega), (show (t + 12) % 15 = 4
212         by omega)] at *
213         norm_num [pat15] at *
214         omega
215     case «8» =>

```

```

202         rw [(show (t + 3) % 15 = 11 by omega), (show (t + 6) % 15 = 14
by omega),
203         (show (t + 9) % 15 = 2 by omega), (show (t + 12) % 15 = 5
by omega)] at *
204         norm_num [pat15] at *
205         omega
206     case «9» =>
207         rw [(show (t + 3) % 15 = 12 by omega), (show (t + 6) % 15 = 0
by omega),
208         (show (t + 9) % 15 = 3 by omega), (show (t + 12) % 15 = 6
by omega)] at *
209         norm_num [pat15] at *
210         omega
211     case «10» =>
212         rw [(show (t + 3) % 15 = 13 by omega), (show (t + 6) % 15 = 1
by omega),
213         (show (t + 9) % 15 = 4 by omega), (show (t + 12) % 15 = 7
by omega)] at *
214         norm_num [pat15] at *
215         omega
216     case «11» =>
217         rw [(show (t + 3) % 15 = 14 by omega), (show (t + 6) % 15 = 2
by omega),
218         (show (t + 9) % 15 = 5 by omega), (show (t + 12) % 15 = 8
by omega)] at *
219         norm_num [pat15] at *
220         omega
221     case «12» =>
222         rw [(show (t + 3) % 15 = 0 by omega), (show (t + 6) % 15 = 3 by
omega),
223         (show (t + 9) % 15 = 6 by omega), (show (t + 12) % 15 = 9
by omega)] at *
224         norm_num [pat15] at *
225         omega
226     case «13» =>
227         rw [(show (t + 3) % 15 = 1 by omega), (show (t + 6) % 15 = 4 by
omega),
228         (show (t + 9) % 15 = 7 by omega), (show (t + 12) % 15 = 10
by omega)] at *
229         norm_num [pat15] at *
230         omega
231     case «14» =>
232         rw [(show (t + 3) % 15 = 2 by omega), (show (t + 6) % 15 = 5 by
omega),
233         (show (t + 9) % 15 = 8 by omega), (show (t + 12) % 15 = 11
by omega)] at *
234         norm_num [pat15] at *
235         omega
236 rw [hperiod n, hr]

```

```

237
238 def pat30 :  $\mathbb{N} \rightarrow \mathbb{Z}$ 
239   | 0 => 1
240   | 1 => 3
241   | 2 => 2
242   | 3 => 0
243   | 4 => 2
244   | 5 => 1
245   | 6 => 1
246   | 7 => 0
247   | 8 => 3
248   | 9 => 0
249   | 10 => 2
250   | 11 => 0
251   | 12 => 2
252   | 13 => 1
253   | 14 => 2
254   | 15 => 1
255   | 16 => 1
256   | 17 => 2
257   | 18 => 2
258   | 19 => 2
259   | 20 => 1
260   | 21 => 3
261   | 22 => 2
262   | 23 => 1
263   | 24 => 2
264   | 25 => 2
265   | 26 => 0
266   | 27 => 0
267   | 28 => 1
268   | _ => 0
269
270 set_option maxHeartbeats 800000 in
271 theorem ap3_mod4_period30 (n r :  $\mathbb{N}$ ) (hr : n % 30 = r) :
272   ap3 n % 4 = pat30 r := by
273   have hperiod :  $\forall m : \mathbb{N}, \text{ap3 } m \% 4 = \text{pat30 } (m \% 30)$  := by
274     intro m
275     induction m using Nat.strong_induction_on with
276     | h m ih =>
277       by_cases hm : m < 12
278       · interval_cases m <=> decide
279       · let t := m - 12
280         have hmt : m = t + 12 := by
281           dsimp [t]
282           omega
283         rw [hmt, ap3]
284         have h0 := ih t (by omega)
285         have h3 := ih (t + 3) (by omega)

```

```

286     have h6 := ih (t + 6) (by omega)
287     have h9 := ih (t + 9) (by omega)
288     have hresBound : t % 30 < 30 := Nat.mod_lt _ (by omega)
289     interval_cases hres : t % 30
290     case «0» =>
291         rw [(show (t + 3) % 30 = 3 by omega), (show (t + 6) % 30 = 6 by
292             omega),
293             (show (t + 9) % 30 = 9 by omega), (show (t + 12) % 30 = 12
294             by omega)] at *
295         norm_num [pat30] at *
296         omega
297     case «1» =>
298         rw [(show (t + 3) % 30 = 4 by omega), (show (t + 6) % 30 = 7 by
299             omega),
300             (show (t + 9) % 30 = 10 by omega), (show (t + 12) % 30 = 13
301             by omega)] at *
302         norm_num [pat30] at *
303         omega
304     case «2» =>
305         rw [(show (t + 3) % 30 = 5 by omega), (show (t + 6) % 30 = 8 by
306             omega),
307             (show (t + 9) % 30 = 11 by omega), (show (t + 12) % 30 = 14
308             by omega)] at *
309         norm_num [pat30] at *
310         omega
311     case «3» =>
312         rw [(show (t + 3) % 30 = 6 by omega), (show (t + 6) % 30 = 9 by
313             omega),
314             (show (t + 9) % 30 = 12 by omega), (show (t + 12) % 30 = 15
315             by omega)] at *
316         norm_num [pat30] at *
317         omega
318     case «4» =>
319         rw [(show (t + 3) % 30 = 7 by omega), (show (t + 6) % 30 = 10
320             by omega),
321             (show (t + 9) % 30 = 13 by omega), (show (t + 12) % 30 = 16
322             by omega)] at *
323         norm_num [pat30] at *
324         omega
325     case «5» =>
326         rw [(show (t + 3) % 30 = 8 by omega), (show (t + 6) % 30 = 11
327             by omega),
328             (show (t + 9) % 30 = 14 by omega), (show (t + 12) % 30 = 17
329             by omega)] at *
330         norm_num [pat30] at *
331         omega
332     case «6» =>
333         rw [(show (t + 3) % 30 = 9 by omega), (show (t + 6) % 30 = 12
334             by omega),

```

```

322         (show (t + 9) % 30 = 15 by omega), (show (t + 12) % 30 = 18
by omega)] at *
323         norm_num [pat30] at *
324         omega
325     case «7» =>
326         rw [(show (t + 3) % 30 = 10 by omega), (show (t + 6) % 30 = 13
by omega),
327             (show (t + 9) % 30 = 16 by omega), (show (t + 12) % 30 = 19
by omega)] at *
328         norm_num [pat30] at *
329         omega
330     case «8» =>
331         rw [(show (t + 3) % 30 = 11 by omega), (show (t + 6) % 30 = 14
by omega),
332             (show (t + 9) % 30 = 17 by omega), (show (t + 12) % 30 = 20
by omega)] at *
333         norm_num [pat30] at *
334         omega
335     case «9» =>
336         rw [(show (t + 3) % 30 = 12 by omega), (show (t + 6) % 30 = 15
by omega),
337             (show (t + 9) % 30 = 18 by omega), (show (t + 12) % 30 = 21
by omega)] at *
338         norm_num [pat30] at *
339         omega
340     case «10» =>
341         rw [(show (t + 3) % 30 = 13 by omega), (show (t + 6) % 30 = 16
by omega),
342             (show (t + 9) % 30 = 19 by omega), (show (t + 12) % 30 = 22
by omega)] at *
343         norm_num [pat30] at *
344         omega
345     case «11» =>
346         rw [(show (t + 3) % 30 = 14 by omega), (show (t + 6) % 30 = 17
by omega),
347             (show (t + 9) % 30 = 20 by omega), (show (t + 12) % 30 = 23
by omega)] at *
348         norm_num [pat30] at *
349         omega
350     case «12» =>
351         rw [(show (t + 3) % 30 = 15 by omega), (show (t + 6) % 30 = 18
by omega),
352             (show (t + 9) % 30 = 21 by omega), (show (t + 12) % 30 = 24
by omega)] at *
353         norm_num [pat30] at *
354         omega
355     case «13» =>
356         rw [(show (t + 3) % 30 = 16 by omega), (show (t + 6) % 30 = 19
by omega),

```

```

357         (show (t + 9) % 30 = 22 by omega), (show (t + 12) % 30 = 25
by omega)] at *
358         norm_num [pat30] at *
359         omega
360     case «14» =>
361         rw [(show (t + 3) % 30 = 17 by omega), (show (t + 6) % 30 = 20
by omega),
362             (show (t + 9) % 30 = 23 by omega), (show (t + 12) % 30 = 26
by omega)] at *
363         norm_num [pat30] at *
364         omega
365     case «15» =>
366         rw [(show (t + 3) % 30 = 18 by omega), (show (t + 6) % 30 = 21
by omega),
367             (show (t + 9) % 30 = 24 by omega), (show (t + 12) % 30 = 27
by omega)] at *
368         norm_num [pat30] at *
369         omega
370     case «16» =>
371         rw [(show (t + 3) % 30 = 19 by omega), (show (t + 6) % 30 = 22
by omega),
372             (show (t + 9) % 30 = 25 by omega), (show (t + 12) % 30 = 28
by omega)] at *
373         norm_num [pat30] at *
374         omega
375     case «17» =>
376         rw [(show (t + 3) % 30 = 20 by omega), (show (t + 6) % 30 = 23
by omega),
377             (show (t + 9) % 30 = 26 by omega), (show (t + 12) % 30 = 29
by omega)] at *
378         norm_num [pat30] at *
379         omega
380     case «18» =>
381         rw [(show (t + 3) % 30 = 21 by omega), (show (t + 6) % 30 = 24
by omega),
382             (show (t + 9) % 30 = 27 by omega), (show (t + 12) % 30 = 0
by omega)] at *
383         norm_num [pat30] at *
384         omega
385     case «19» =>
386         rw [(show (t + 3) % 30 = 22 by omega), (show (t + 6) % 30 = 25
by omega),
387             (show (t + 9) % 30 = 28 by omega), (show (t + 12) % 30 = 1
by omega)] at *
388         norm_num [pat30] at *
389         omega
390     case «20» =>
391         rw [(show (t + 3) % 30 = 23 by omega), (show (t + 6) % 30 = 26
by omega),

```



```

392         (show (t + 9) % 30 = 29 by omega), (show (t + 12) % 30 = 2
by omega)] at *
393         norm_num [pat30] at *
394         omega
395     case «21» =>
396         rw [(show (t + 3) % 30 = 24 by omega), (show (t + 6) % 30 = 27
by omega),
397             (show (t + 9) % 30 = 0 by omega), (show (t + 12) % 30 = 3
by omega)] at *
398         norm_num [pat30] at *
399         omega
400     case «22» =>
401         rw [(show (t + 3) % 30 = 25 by omega), (show (t + 6) % 30 = 28
by omega),
402             (show (t + 9) % 30 = 1 by omega), (show (t + 12) % 30 = 4
by omega)] at *
403         norm_num [pat30] at *
404         omega
405     case «23» =>
406         rw [(show (t + 3) % 30 = 26 by omega), (show (t + 6) % 30 = 29
by omega),
407             (show (t + 9) % 30 = 2 by omega), (show (t + 12) % 30 = 5
by omega)] at *
408         norm_num [pat30] at *
409         omega
410     case «24» =>
411         rw [(show (t + 3) % 30 = 27 by omega), (show (t + 6) % 30 = 0
by omega),
412             (show (t + 9) % 30 = 3 by omega), (show (t + 12) % 30 = 6
by omega)] at *
413         norm_num [pat30] at *
414         omega
415     case «25» =>
416         rw [(show (t + 3) % 30 = 28 by omega), (show (t + 6) % 30 = 1
by omega),
417             (show (t + 9) % 30 = 4 by omega), (show (t + 12) % 30 = 7
by omega)] at *
418         norm_num [pat30] at *
419         omega
420     case «26» =>
421         rw [(show (t + 3) % 30 = 29 by omega), (show (t + 6) % 30 = 2
by omega),
422             (show (t + 9) % 30 = 5 by omega), (show (t + 12) % 30 = 8
by omega)] at *
423         norm_num [pat30] at *
424         omega
425     case «27» =>
426         rw [(show (t + 3) % 30 = 0 by omega), (show (t + 6) % 30 = 3 by
omega),

```

```

427         (show (t + 9) % 30 = 6 by omega), (show (t + 12) % 30 = 9
by omega)] at *
428         norm_num [pat30] at *
429         omega
430     case «28» =>
431         rw [(show (t + 3) % 30 = 1 by omega), (show (t + 6) % 30 = 4 by
omega),
432             (show (t + 9) % 30 = 7 by omega), (show (t + 12) % 30 = 10
by omega)] at *
433         norm_num [pat30] at *
434         omega
435     case «29» =>
436         rw [(show (t + 3) % 30 = 2 by omega), (show (t + 6) % 30 = 5 by
omega),
437             (show (t + 9) % 30 = 8 by omega), (show (t + 12) % 30 = 11
by omega)] at *
438         norm_num [pat30] at *
439         omega
440 rw [hperiod n, hr]

```

## 4 Verification evidence

- Toolchain: Lean v4.34.0 (leanprover/lean4:v4.34.0), mathlib4 v4.34.0, revision 5ed29652. Reproduction: with that pin, lake env lean on proofs/01-comb10-ladder-pend3-proved.lean compiles with exit code 0 and empty output.
- Statement integrity: the frozen statement file (160 lines) has sha256

47d54db7c3d5309720537295055d5869e990618  
399950aaacb311f8028a9bba9,

and the final proof file (440 lines) has sha256

e56e496f8ddc8310316a1d6f1f9befa17e76200  
f97b22515dcd0b2260e9fd7b1.

A re-run of the audit department’s symmetric-difference method (comments stripped, declarations split, definitions compared full-block, theorems up to :=) reports all nine frozen declarations header-identical, with exactly the three auxiliary lemmas `ap3_sub_0/1/2` added as proof-body scaffolding (`audit/statement-diff.out`).

- Kernel check: compiles with 0 `sorry` (whole-file scan including comments, also covering `admit`: zero hits); `#print axioms` output, verbatim (kernel re-run of 2026-09-29, exit code 0, kept in `audit/axioms.out`):

```

1 'ap3_interleave' depends on axioms: [propext, Quot.sound]
2 'ap3_mod2_period15' depends on axioms: [propext, Classical.choice,
    Quot.sound]
3 'ap3_mod4_period30' depends on axioms: [propext, Classical.choice,
    Quot.sound]

```

Each line is a subset of the whitelist {`propext`, `Quot.sound`, `Classical.choice`}.

- Grading by the audit department: all three proven declarations *complete proof*; the lowest grade reached for the case = *complete proof* (final adjudication `final-comb10-ladder-pend3.txt`: statement 9/9 byte-identical, strict compile exit 0, zero `sorry/admit`, axiom whitelist subset). No weakening, no added hypotheses, no unproven residue.
- Audit chain (originals in this deposit under `audit/` unless noted): the final adjudication, the statement-difference re-run, the axiom re-print, this note’s claim check `claim-check.md` — added to `audit/` after the audit department returns its adjudication, and a prerequisite for publication.

## 5 Novelty evidence

Adjudication copied from the pipeline’s exclusivity reviews (C9 gate), which ran before the problem was accepted: verdict **no occupying record found** for the period-3 pendant-ladder matching sequence, its order-12 sparse recurrence, and its three mod-3 subsequences — at both the object level (miner screen) and the proposition level (independent full review), confirmed by a second review that deliberately reused no query method of the first. Value tier **new sequence / new recurrence**. The registered family members are delimited explicitly and none of them is this object: the plain ladder A030186 [4], the period-1 pendant corona A102436 [5], the period-2 pendant A386889 [6]; the nearest literature neighbour, Farrell’s “Matchings in ladders” [7], covers the plain ladder only. The claim is strictly *new sequence / new recurrence*, never new mathematics. Summary by channel; the full query log, including every zero-hit query, is in this deposit at `audit/c9-record.md` (two records, byte-for-byte).

| channel                  | period                                                                                                                                                                                     | note                                                                                                                                                   |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| OEIS: numeric 3d         | first review 10 (4 prime windows, 25 $\times$ 17 $\times$ 11, three subsequence $\times$ 3 terms) + second review 8 (all-but-prime obj(17)10, min(deg(1st window), subsequence 1st window) | all zero bits; positive-control markers for (P-headers, A180896, finally forward empty)                                                                |
| OEIS: recursive/term     | first review 9 (prime-sum, coefficient, generating function, characteristic polynomial in two-variables) + second review 4 (OEIS-entry-attach type)                                        | all zero bits                                                                                                                                          |
| OEIS: keyword/label      | first review 12 + second review 8 (fully-defective ladder, three-receiver phasing, independent obj wts) + 2 also-sought with strings (Chinese layer, discrete)                             | note bits only, since the object, reverse cross-reference hunting; A260889 has no reverse references, the A000046 citing family has no period-3 member |
| OEIS: factor + 3-adj/ord | first review: 1000 APF rate-based, 2 also-sought sub-search results, second review: 3 to-one strings + 3-adj/ord/ord (1 query + 2 subqueries)                                              | contains bit (testing-emptiness ladder third), the object absent                                                                                       |
| addMTH                   | first review 5 + second review 5 + full-forward end                                                                                                                                        | new bits for the object: Rowell 1978 (as-0114-17042) read in full: photo header, no pendant construction                                               |
| asXto APF                | first review 6 + second review 6, with estimate                                                                                                                                            | contains bit: current neighbors as provided building, spin-ladder physics, primitive dimerity — none the object                                        |
| OpenAble                 | 8 bit, 100-bit/word                                                                                                                                                                        | no this-object tick in any entry                                                                                                                       |
| Measure layer            | switch to, complete, separable two (20000 bits)                                                                                                                                            | any graph-header or pendant content; matches has matching predicate but no counting APF                                                                |
| Independent to count     | batch implementation (unknown-number representation)                                                                                                                                       | 30 30 agreement with the deep-46 log, guards against single-implementation blind spots                                                                 |

The channels are named so a later reader can re-run them; if any channel later returns a same-form record, the tier above weakens accordingly, and this deposit will be amended by a later version rather than by editing this record.

## 6 Scope and limitations

Grade and boundary, stated as the audit states them. (a) The kernel layer proves the pure-sequence theorems Theorems 1–3 and nothing else. (b) The combinatorial identification of **ap3** with the matching count of the period-3 pendant ladder is *not* kernel-proved: it rests on the pipeline’s phase-0 computational evidence (five methodologically distinct counting implementations agreeing over 241 terms; double-window modular scans; rational-domain least-order fitting), is carried at the conjecture / report layer, and does not become a theorem by this deposit. (c) The minimality of the periods 15 and 30 is numerically observed (first-return state scan, unique minimum within 20000 terms) but not kernel-proved here; the theorems assert the residue laws for the tables as written. (d) Two further modular laws (period 39 mod 3, period 60 mod 8) and a pure-period existence statement for every modulus are numerically supported but not kernel-proved and are not claimed. (e) The perfect-matching (PM) subsequence has zero-interleaved, too-short non-zero segments for effective sequence search; it was not novelty-searched and is not claimed. (f) The gate-four sign-off of the underlying task was exercised by the author’s instruction of 2026-09-29 to run both the paper lane and this claim lane with direct outbound authorisation, while the task report file header retains its pre-sign-off wording unchanged; both calibers are recorded. (g) This note carries no literature review and is a deposit record, not a survey; the narrative paper is prepared separately under the pipeline’s paper lane and will reference this deposit.

## Data availability

The deposit package for this claim contains the claim note (LaTeX source and PDF), the Lean sources (**proofs/**: frozen statement file, final proof file, **lean-toolchain**), the verification and audit logs, and the full C9 query log with zero-hit entries (**audit/**). Public mirror: the claim lane’s

repository <https://github.com/Aurora0134/ai4math-claims>, directory `ladder-pend3-interleave/` (pushed under the author’s instruction of 2026-09-29; the commit and timestamp are recorded in the source repository’s `claims/ladder-pend3-interleave/card.md`). Zenodo: the DOI is reserved at upload by the author and is recorded in `metadata/README.md` of this package once created; see `UPLOAD.md` in the source repository. Claim note under CC BY 4.0, code under MIT.

## Use of generative AI

(a) AI performed: candidate generation and the two-stage exclusivity review, autoformalisation of the definitions and statements, the roundtrip semantic check, proof search and proof-script writing (worker agents, with main-agent repair on one theorem after a tactic-level rejection), error classification and repair, and the assembly of this note including its verbatim listings. (b) Model, node and budget: the proving phase ran on the node designated by the author for that session (`a6api/gpt-6-sol`; the end-point/model detection scripts resolved unknown for that session and the designation is recorded in the task ledger’s override log), with no cross-node switching, consuming 7 agent-run samples of a permitted 20, 0 llm-calls of a permitted 8, and 27 strict kernel compiles of a permitted 48 (plus compile-debug probes, separately ledgered), copied verbatim from the task ledger. This note consumed 0 pipeline LLM calls against a cap of 4; drafting was mechanical assembly. (c) Final adjudication: all statements and proofs reported here were checked by the Lean 4 kernel: the proof compiles with no `sorry`, and `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. (d) The AI system is not an author; the human author reviewed the evidence and is responsible for all content.

## Author contributions

CRediT-style: the human author adjudicated topic selection, statement semantics and the deposit decision; the AI4Math pipeline (an LLM) generated the candidate propositions, the proof searches and the text drafts. The AI system is not listed as an author.

## References

- [1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *Automated Deduction – CADE-28*, LNCS, pp. 625–635, 2021. [https://doi.org/10.1007/978-3-030-79876-5\\_37](https://doi.org/10.1007/978-3-030-79876-5_37)
- [2] The mathlib Community. The Lean mathematical library. *CPP 2020*, pp. 367–381, 2020. <https://doi.org/10.1145/3372885.3373824>
- [3] The OEIS Foundation. *The On-Line Encyclopedia of Integer Sequences*. <https://oeis.org/>
- [4] OEIS Foundation. Matchings of the ladder graph  $P_2 \times P_n$  (%N line gives the defining recurrence  $a(n) = 3a(n-1) + a(n-2) - a(n-3)$ ,  $a(0) = 1$ ,  $a(1) = 2$ ,  $a(2) = 7$ ). *The On-Line Encyclopedia of Integer Sequences*, entry A030186. <https://oeis.org/A030186>
- [5] OEIS Foundation. Number of matchings of the corona  $L'(n)$  of the ladder graph  $L(n) = P_2 \times P_n$  and the complete graph  $K(1)$  (the period-1 pendant relative). *The On-Line Encyclopedia of Integer Sequences*, entry A102436. <https://oeis.org/A102436>
- [6] OEIS Foundation. Number of ways to use  $1 \times 1$  squares and  $1 \times 2$  dominos (in any orientation) to tile a  $3 \times (2n-1)$  strip with every other cell in the top row removed (the period-2 pendant relative). *The On-Line Encyclopedia of Integer Sequences*, entry A386889. <https://oeis.org/A386889>
- [7] E. J. Farrell. Matchings in ladders. *Ars Combinatoria* 6 (1978), pp. 153–161. (zbMATH an:0414.05042.)